Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Coding standards - purpose

- They play a crucial role in software development:
  - consistency: ensure uniformity across codebases, making it easier for devs to read, understand and maintain code
  - readability: code readability is enhance through standards, reducing errors and improving collaboration
  - error prevention: help catch common mistakes early, preventing bug
  - scalability: code can scale without becoming unwieldy or unmanageable
  - cross-team collaboration: improve collaboration among developers, even in large teams
  - code reviews: more effective feedback
  - efficient maintenance: simplifies debugging, refactoring and maintenance task

Choose Industry-specific coding standards

- Coding best practices and standards vary depending on the industry a specific product is being built for.
- The standards required for coding software for luxury automobiles will differ from those for gaming software.
- For example, MISRA C and C++ were written for the automotive industry and are considered the de-facto standards for building applications that emphasize safety. They are the absolute best practices for writing code in the industry.
- Adhering to industry-specific coding standards in software engineering makes writing correct code that matches product expectations easier. Writing code that will satisfy the end-users and meet business requirements becomes easier.

Ca' Foscari University of Venice

Code readability

- Write as few lines as possible.
- Use appropriate naming conventions.
- Segment blocks of code in the same section into paragraphs.
- Use indentation to mark the beginning and end of control structures. Specify the code between them.
- Do not use lengthy functions. Ideally, a single function should carry out a single task.
- Avoid long lines, it is easier for humans to read blocks of lines
- Use the "Don' Repeat Yourself" principle: automate repetitive tasks whenever necessary. The same piece of code should not be repeated in the script.
- Avoid deep nesting: too many nesting levels make code harder to read and follow
- Capitalize SQL special words and function names to distinguish them from table and column names

Meaningful names

- Choose meaningful names that convey the purpose of the variable or function

```
// Bad
const cust = "Matteo"
const customer = "Anna"
// Better
const customerName = "Matteo"
const customerFullName = "Alice Johnson"
```

- Different naming conventions used in coding:
  - camelCase, commonly used in JS for variables and function names
  - snake_case
  - kebab-case
  - PascalCase, typically used for class names in JS

An identifier for multiple purposes

- the name of each variable has to describes its purpose.
- A single variable can't be assigned various values or utilized for numerous functions.
- Always assign unique variable names.

Add comments and prioritize documentation

When to add comments

- Include comments for intricate or non-obvious code segments.
- Explain business rules, domain-specific logic, or regulatory requirements.
- Clarify how your code handles edge cases or exceptional scenarios.
- Document workarounds due to limitations or external dependencies.
- Mark areas where improvements or additional features are needed.

When not to add comments

- Avoid redundant comments that merely repeat what the code already expresses clearly.
- If the code's purpose is evident (e.g., simple variable assignments), skip unnecessary comments.
- Remove temporary comments used for debugging once the issue is resolved.
- Incorrect comments can mislead other developers, so ensure accuracy.

Other notes

- Efficient data processing – divide code into smaller, self-contained modules or functions for reusability and maintainability.
- Effective version control and collaboration – ensure all devs follow consistent coding techniques.
- Effective code review and refactoring – engage QA during refactoring to prevent new bugs.
- Try to formalize exception handling – keep code in a try-catch block, ensure auto recovery, consider that it might be an issue of software/network slowness and use real-time log analysis

Other notes

- Security and privacy considerations
- Standardize headers for different modules, e.g.:
  - Module name
  - Date of creation
  - Name of creator of the module
  - History of modification
  - Functions in that module
  - …
- Daily backups: multiple events can trigger data loss, save code daily and back up the workflows
- When choosing standards, think Closed vs Open